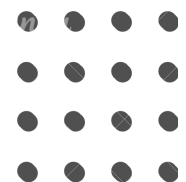




VMES Grinds





Leetcode: 739 Daily Temperatures



Noel



739. Daily Temperatures

Solved

Medium

Topics

Companies

Hint

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `ith` *day to get a warmer temperature.* If there is no future day for which this is possible, keep `answer[i] == 0` instead.



Example 1:

Input: `temperatures = [73,74,75,71,69,72,76,73]`

Output: `[1,1,4,2,1,1,0,0]`

Example 2:

Input: `temperatures = [30,40,50,60]`

Output: `[1,1,1,0]`

Example 3:

Input: `temperatures = [30,60,90]`

Output: `[1,1,0]`

Constraints:

- `1 <= temperatures.length <= 105`
- `30 <= temperatures[i] <= 100`



Explanation





Input: [73, 74, 75, 71, 69, 72, 76, 73]

Step-by-Step Execution

Initial: res = [0, 0, 0, 0, 0, 0, 0, 0]
stack = []

Step 0: i=0, temp=73

Action: Push 0
Stack: [0]
res: [0, 0, 0, 0, 0, 0, 0, 0]

Step 1: i=1, temp=74

Check: 74 > 73? YES
→ Pop 0, res[0] = 1 - 0 = 1
Action: Push 1
Stack: [1]
res: [1, 0, 0, 0, 0, 0, 0, 0]

Step 2: i=2, temp=75

Check: 75 > 74? YES
→ Pop 1, res[1] = 2 - 1 = 1
Action: Push 2
Stack: [2]
res: [1, 1, 0, 0, 0, 0, 0, 0]

Step 3: i=3, temp=71

Check: 71 > 75? NO
Action: Push 3
Stack: [2, 3]
res: [1, 1, 0, 0, 0, 0, 0, 0]

Step 4: i=4, temp=69

Check: 69 > 71? NO
Action: Push 4
Stack: [2, 3, 4]
res: [1, 1, 0, 0, 0, 0, 0, 0]

Step 5: i=5, temp=72

Check: 72 > 69? YES
→ Pop 4, res[4] = 5 - 4 = 1
Check: 72 > 71? YES
→ Pop 3, res[3] = 5 - 3 = 2
Check: 72 > 75? NO
Action: Push 5
Stack: [2, 5]
res: [1, 1, 0, 2, 1, 0, 0, 0]

Step 6: i=6, temp=76

Check: 76 > 72? YES
→ Pop 5, res[5] = 6 - 5 = 1
Check: 76 > 75? YES
→ Pop 2, res[2] = 6 - 2 = 4
Action: Push 6
Stack: [6]
res: [1, 1, 4, 2, 1, 1, 0, 0]

Step 7: i=7, temp=73

Check: 73 > 76? NO
Action: Push 7
Stack: [6, 7]
res: [1, 1, 4, 2, 1, 1, 0, 0]

Final Result

res = [1, 1, 4, 2, 1, 1, 0, 0]

Remaining indices in stack [6, 7] have no warmer day → already 0



Initial Approach





1. Initialize a stack (to store the indexes for counting days) and ans for storing results
2. Loop through the temperatures
3. Check if today's temperature is greater than the day at the top of the stack
4. If greater, pop from the stack and calculate days and update ans
5. Push today's temperatures and move on
6. Return the ans at the end
7. Done!



1. Initialize a stack (to store the indexes for counting days) and ans for storing results
2. Loop through the temperatures
3. Check if today's temperature is greater than the day at the top of the stack
4. If greater, pop from the stack and calculate days and update ans
5. Push today's temperatures and move on
6. Return the ans at the end
7. Done!

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) → List[int]:
        n = len(temperatures)
        ans = [0] * n
        stack = []

        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                prev = stack.pop()
                days = i - prev
                ans[prev] = days

            stack.append(i)

        return ans
```



Time and Space Complexity Analysis





Space - $O(n)$

Time - $O(n)$

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) → List[int]:
        n = len(temperatures)
        ans = [0] * n
        stack = []

        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                prev = stack.pop()
                days = i - prev
                ans[prev] = days

            stack.append(i)

        return ans
```



Key Takeaway



1. Stack maintains decreasing temperatures (monotonic stack pattern)
2. Pop and resolve when current temp breaks the decreasing order
3. Remaining indices in stack have no answer (keep as 0)